



ALGORITMIA

Entrega P0-P7

Samuel Tobías Menéndez

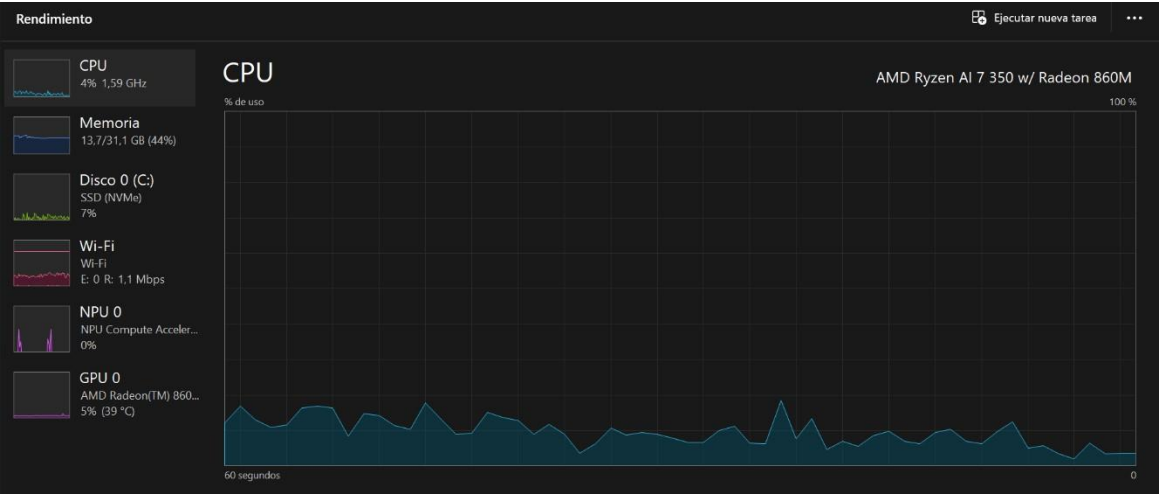
UO305955

Índice

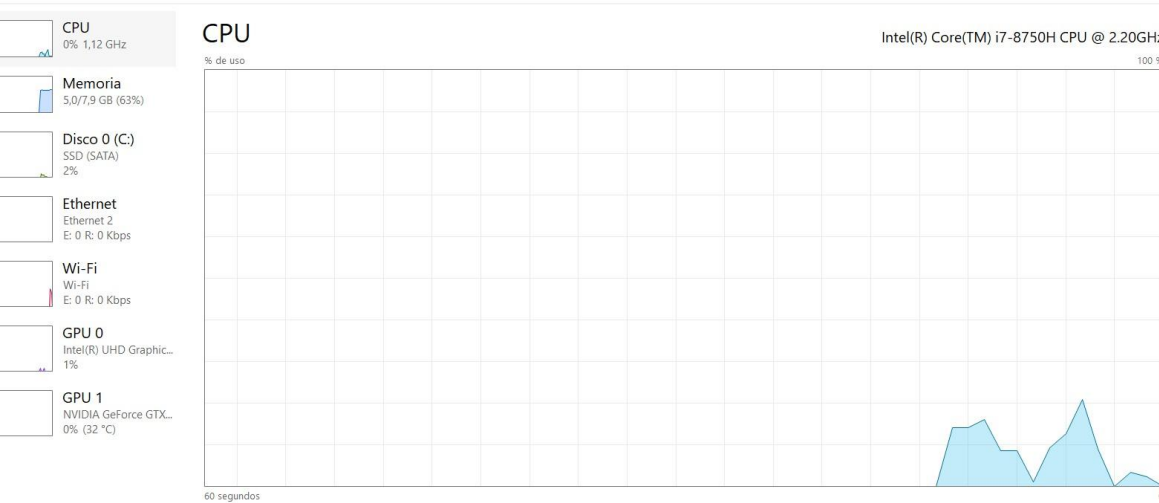
Práctica 0	2
Práctica 1-1.....	5
Práctica 1-2	14
Práctica 2	19
Práctica 3	28
Práctica 4	36
Práctica 5	38
Práctica 6	39
Práctica 7	41

Práctica 0

Equipo 1:



Equipo 2:



PythonA1

Equipo 1:

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	742	1861	7129	32216	155768

Equipo 2:

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	748	3203	12192	47627	191452

JavaA1

Sin
Optimizació
n

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	195	426	1413	5555	22266

Con Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	80	194	429	1526	5079

JavaA1 – Eclipse

Con Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	41	99	442	1272	5137

Sin Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	184	442	1413	5474	21513

PythonA2

Equipo 1:

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	74	249	769	2843	10575

JavaA2

Con Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	21	14	48	150	575

Sin Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	39	53	185	607	2094

PythonA3

Equipo 1:

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	72	144	425	1439	5423

JavaA3

Con Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	58	12	41	142	300

Sin Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	43	41	104	394	1228

PythonA4

Equipo 1:

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	2	4	7	16	32

JavaA4

Con Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	36	1	2	3	1

Sin Optimización

Valores	5000	10000	20000	40000	80000
Tiempo(ms)	0	2	4	4	4

Práctica 1-1

¿Calcular cuántos años más podremos seguir utilizando esta forma de contar? Explica el razonamiento seguido para realizar el cálculo.

En 64 bits se pueden representar 2^{64} números, 1.8446744×10^{19} milisegundos, lo que es igual a 5.849.424.150.177.575 años. Si le restamos los 56 años que han pasado desde el 70 obtenemos la solución.

$5,8 \times 10^8 - 56$ años hasta que llegue al tope la cuenta.

¿Por qué a veces el tiempo medido sale 0? ¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables?

Aunque en mis cálculos no pasa, a veces puede salir tiempos muy inferiores o 0 debido a usar la optimización. La optimización contiene el JIT que no interpreta el código línea a línea y debido a esto salta líneas innecesarias para la salida.

Si consideramos tiempos por debajo de 50ms despreciables, a partir de los 10mill ya son tiempos fiables.

Repeticiones	1mill	10mill	20mill	50mill	100mill	200mill
Tiempo (ms)	8	74	153	312	653	1469

Vector 3

```

n    tTiempo
10000  0
20000  0
40000  0
80000  0
160000 0
320000 0
640000 7
1280000 16
2560000 15
5120000 43
10240000 56
20480000 126
40960000 249
81920000 515

Fin de la medicion de tiempos *****

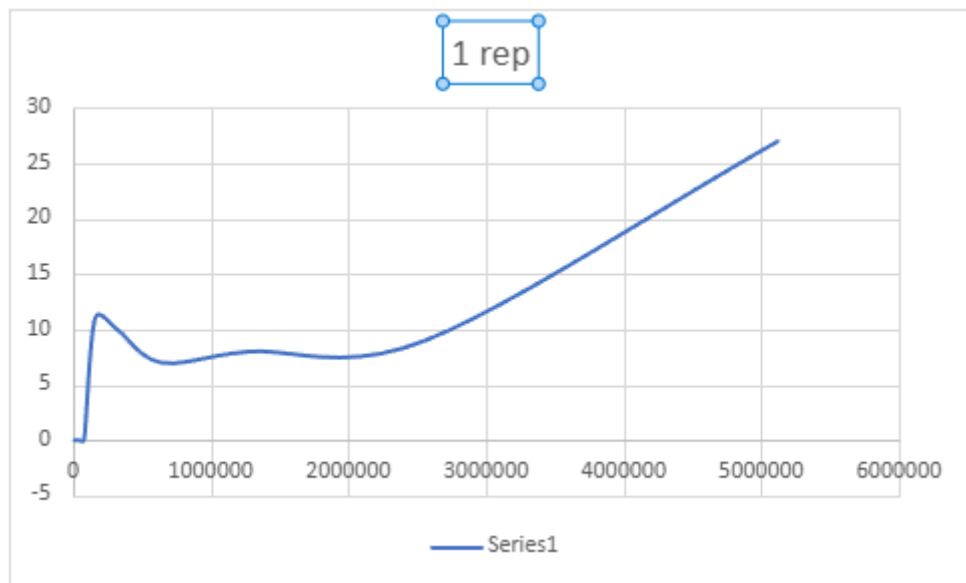
```



Vector 4

1 repetición:

n	Tiempo
10000	0
20000	0
40000	0
80000	0
160000	11
320000	10
640000	7
1280000	8
2560000	9
5120000	27



10 repeticiones:

repeticiones = 10	
n	Tiempo
10000	13
20000	0
40000	18
80000	6
160000	17
320000	22
640000	50
1280000	97
2560000	157
5120000	316



100 repeticiones:

```
alg_TobiasMenendezSamu
repeticiones = 100
n      Tiempo
10000  18
20000  29
40000  43
80000  70
160000 133
320000 228
640000 395
1280000 794
2560000 1588
5120000 3175
```



1000 repeticiones:

```
repeticiones = 1000
n      Tiempo
10000  88
20000  153
40000  266
80000  541
160000 1094
320000 2015
640000 3985
1280000 8042
2560000 16347
5120000 32695
```



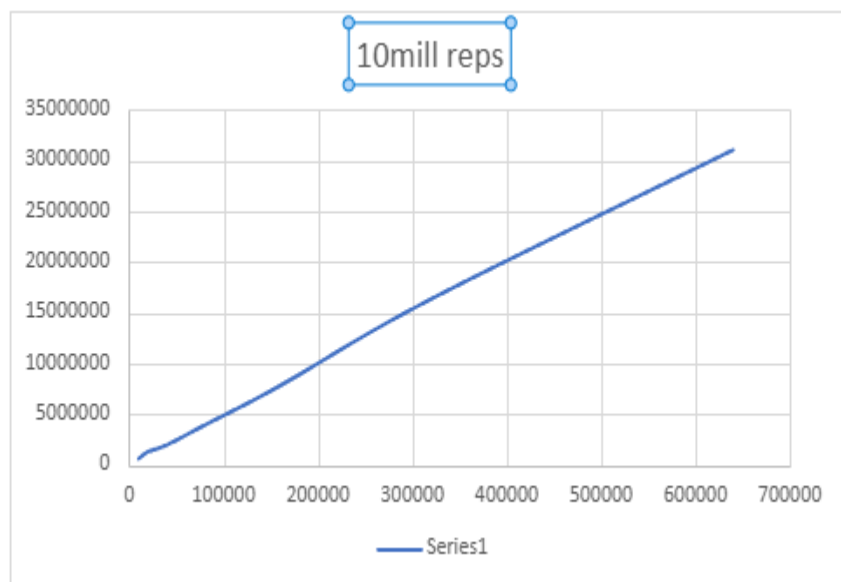
100000 repeticiones:

```
repeticiones = 100000
n      Tiempo
10000  7048
20000  18149
40000  28913
80000  48710
160000 97283
320000 196079
640000 391706
1280000 789586
2560000 1571810
5120000 4379122
```



10mill de repeticiones (hago hasta 640000 por que lleva un día para hacer hasta lo medido y para lo siguiente iba a tardar mucho más):

```
repeticiones = 10000000
n      Tiempo
10000  599822
20000  1298124
40000  1966022
80000  3922145
160000 7813192
320000 16367103
640000 31076432
```



¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2? ¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para $k=3$ y $k=4$ y compruebe los tiempos obtenidos.)

Para dar respuesta a esta pregunta voy a usar los tiempos obtenidos con 1000 repeticiones como referencia.

```

repeticiones = 1000
n      Tiempo
10000   88
20000  153
40000  266
80000  541
160000 1094
320000 2015
640000 3985
1280000 8042
2560000 16347
5120000 32695

```

```

repeticiones = 2000
n      Tiempo
10000  116
20000  257
40000  470
80000  893
160000 1696
320000 3271
640000 6472
1280000 12774
2560000 25235
5120000 50383

```

```

repeticiones = 3000
n      Tiempo
10000  162
20000  285
40000  584
80000  1168
160000 2277
320000 4636
640000 9331
1280000 18903
2560000 37772
5120000 75423

```

```

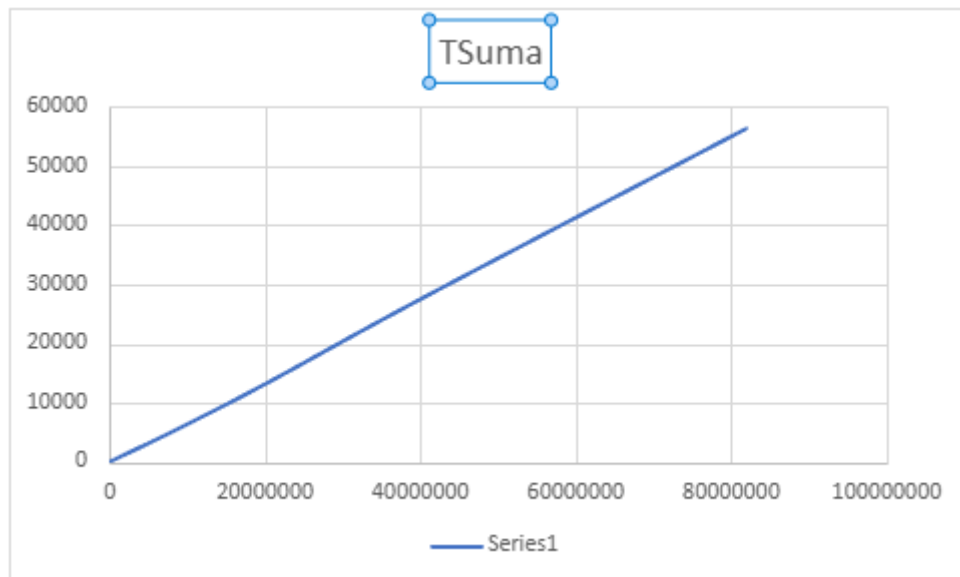
repeticiones = 4000
n      Tiempo
10000  220
20000  374
40000  773
80000  1552
160000 3047
320000 6196
640000 12357
1280000 25087
2560000 50178
5120000 99419

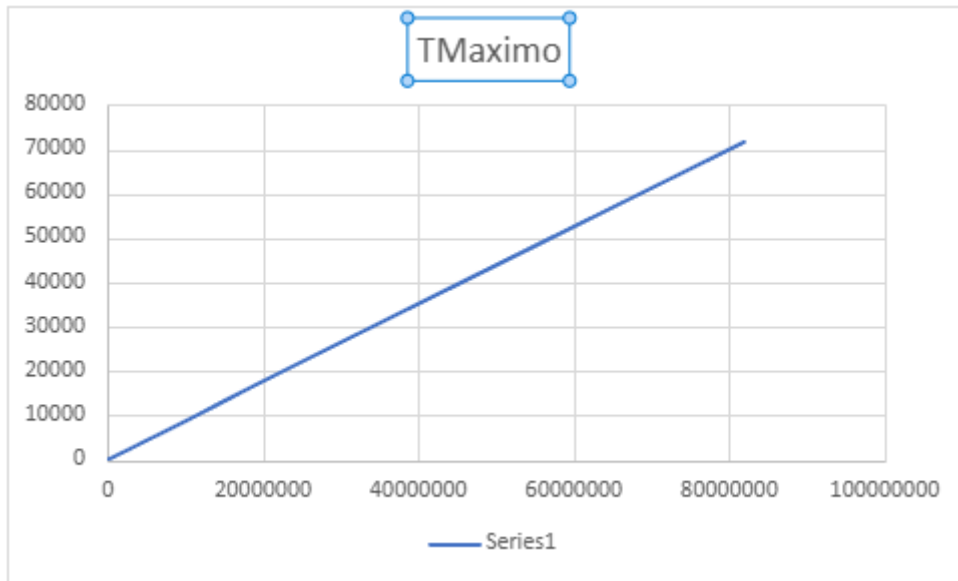
```

Aproximadamente, cuando se multiplican las repeticiones por una constante, el tiempo también lo hace.

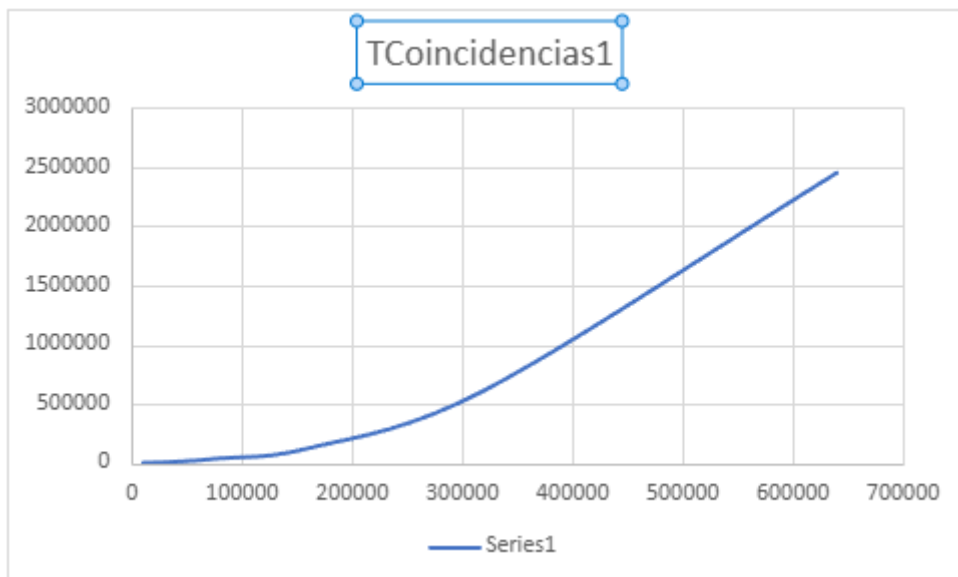
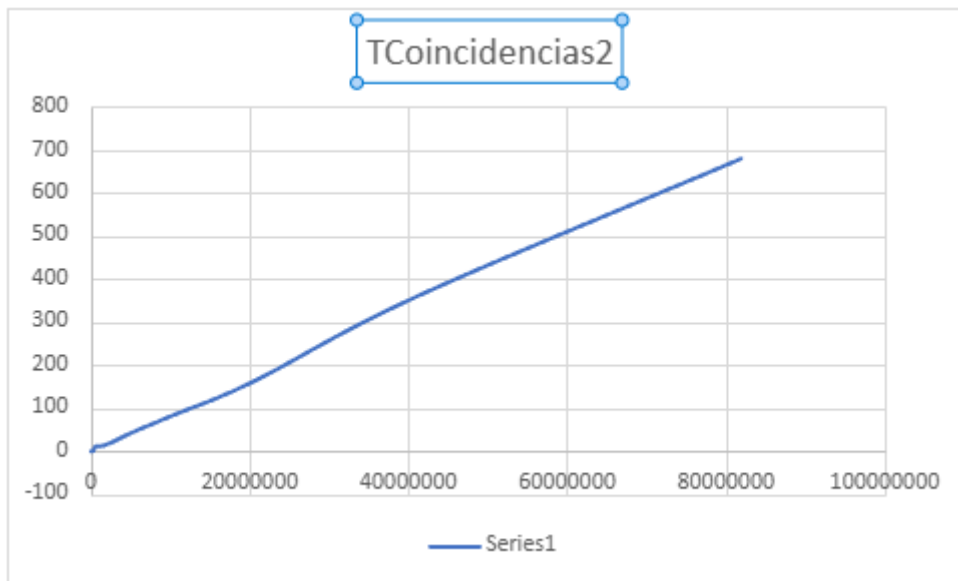
A partir de lo visto en Vector4.java midiendo los tiempos de suma, realice las tres siguientes Clases: Vector5.java para medir los tiempos del máximo, Vector6.java para medir los tiempos de coincidencias1 y Vector7.java para medir los tiempos de coincidencias2.

n	TSuma	TMaximo
10000	14	20
20000	18	29
40000	33	49
80000	62	93
160000	128	167
320000	213	288
640000	405	553
1280000	817	1129
2560000	1620	2213
5120000	3216	4469
10240000	6529	8935
20480000	13489	18159
40960000	28183	36030
81920000	56353	71658





n	TCoincidencias1	TCoincidencias2
10000	675	0
20000	2420	0
40000	9582	0
80000	38282	0
160000	123081	0
320000	612996	0
640000	2452972	12
1280000	Fdt	12
2560000	Fdt	20
5120000	Fdt	43
10240000	Fdt	83
20480000	Fdt	162
40960000	Fdt	359
81920000	Fdt	680

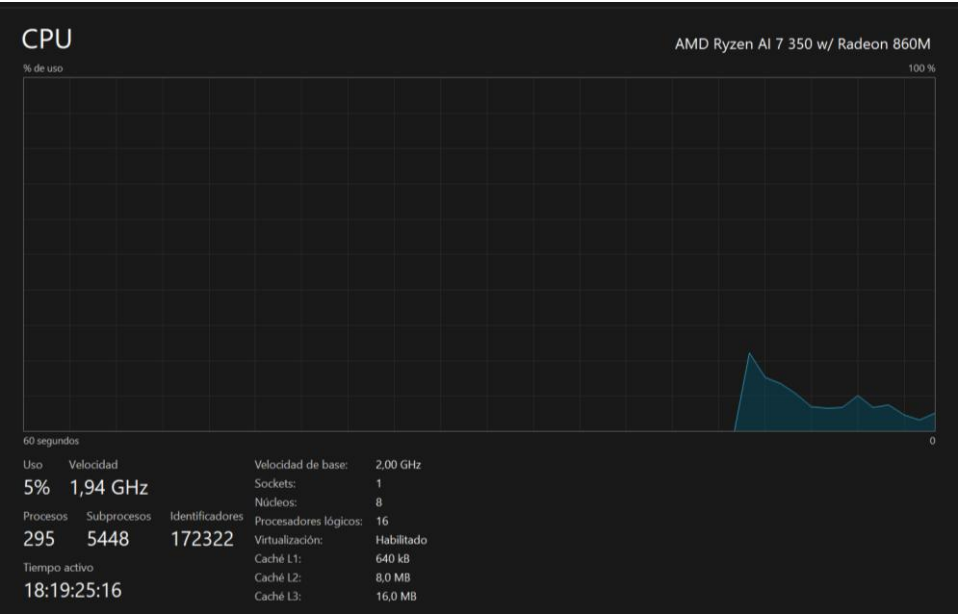


(Aunque la práctica especifique que más de un minuto se considera fuera de tiempo, he decidido coger más valores para que se vean mejor los datos obtenidos).

Una vez rellenadas ambas tablas, concluya si los tiempos obtenidos sintonizan con lo esperado, según la complejidad temporal computacional de las diferentes operaciones.

Los resultados concuerdan con lo esperado. $T_{\text{máximo}}$, T_{Suma} , $T_{\text{Coincidencias2}}$ son $O(n)$ mientras que $T_{\text{Coincidencias1}}$ es $O(n^2)$.

Indique las características principales (procesador, memoria) del ordenador donde se han medido tiempos.



Práctica 1-2

Bucle 1:

```
ndeZSamuelU0305955\p12> java -Xint p12.Bucle1 1
```

n	tiempo	repeticiones	contador
100	0	1	603
200	0	1	1340
400	0	1	2937
800	0	1	6942
1600	0	1	14938
3200	7	1	32010
6400	2	1	68272
12800	0	1	153612
25600	0	1	324273
51200	9	1	682680

Bucle 2:

n	tiempo	repeticiones	contador
100	0	1	25500
200	0	1	101000
400	14	1	482400
800	36	1	2245600
1600	116	1	8971200
3200	304	1	40985600
6400	1096	1	163891200
12800	4859	1	737395200
25600	27209	1	3277056000
51200	101758	1	13107712000

Bucle 3:

n	tiempo	repeticiones	contador
100	0	1	71400
200	9	1	323200
400	19	1	1447200
800	83	1	6416000
1600	286	1	28195200
3200	1093	1	122956800
6400	5009	1	532646400
12800	30103	1	2294118400
25600	100245	1	9831168000
51200	372059	1	41944678400

Bucle 4:

n	tiempo	repeticiones	contador
100	2	1	171700
200	15	1	1353400
400	71	1	10746800
800	451	1	85653600
1600	3255	1	683947200
3200	29199	1	5466454400
6400	223960	1	43711148800

n	TBucle1	TBucle2	TBucle3	TBucle4
100	0	0	0	2
200	0	0	9	15
400	0	14	19	71
800	0	36	83	451
1600	0	116	286	3255
3200	7	304	1093	29199
6400	2	1096	5009	223960
12800	0	4859	30103	Fdt
25600	0	27209	100245	Fdt
51200	9	101759	372059	Fdt

Las complejidades de los anteriores bucles son los siguientes:

Bucle1: $2n * \log(n^2)$ que si se simplifica es $2n*2\log(n)$ luego complejidad $O(n*\log(n))$.

Bucle2: $\log(n) * n * (n/2)$ que si se simplifica es $\log(n)*(n^2/2)$ luego complejidad $O(n^2 * \log(n))$.

Bucle3: $n * (n/2) * \log(n)$ que es equivalente a $\log(n)*(n^2/2)$ luego complejidad $O(n^2*\log(n))$. La diferencia de tiempos que hay entre este bucle y el anterior es que la variable del segundo bucle de este depende del valor del primer bucle.

Bucle4: $n*n*n$ que es complejidad $O(n^3)$.

Viendo los tiempos recogidos en la tabla anterior los tiempos cuadran con las complejidades.

Bucle5:

n	tiempo	repeticiones	contador
100	0	1	84379
200	11	1	453287
400	30	1	2360752
800	133	1	11968002
1600	563	1	59294685
3200	2484	1	288106857
6400	11636	1	1376928502

Bucle 6:

n	tiempo	repeticiones	contador
100	3	1	805903
200	109	1	7594028
400	508	1	70524793
800	4088	1	645343578
1600	36569	1	5826168107
3200	334458	1	51983597292

Bucle 7:

n	tiempo	repeticiones	contador
100	30	1	4421275
200	409	1	68685050
400	5278	1	1082740100
800	88876	1	17194960200

n	TBucle5	TBucle6	TBucle7
100	0	3	30
200	11	109	409
400	30	508	5278
800	133	4088	88876
1600	563	36569	FdT
3200	2484	334458	FdT
6400	11636	FdT	FdT
12800	FdT	FdT	FdT
25600	FdT	FdT	FdT
51200	FdT	FdT	FdT

Las complejidades son las siguientes:

Bucle5: $O(n^2 \log^2 n)$

Bucle6: $O(n^3 \log n)$

Bucle7: $O(n^4)$

Los tiempos concuerdan con las complejidades. Bucle7 es el que tiene mayores tiempos y complejidad, Bucle6 el segundo y Bucle5 el menor tanto en tiempo como en complejidad.

Para la siguiente tabla donde el tiempo me da 0 milisegundos, pongo 1 para que pueda hacer los cálculos

n	TBucle(t1)	TBucle(t2)	t1/t2
100	1	1	1
200	1	1	1
400	1	14	0,0714
800	1	36	0,0278
1600	1	116	0,0086
3200	7	304	0,0230
6400	2	1096	0,0018
12800	1	4859	0,0002
25600	1	27209	0,00003
51200	9	101758	0,00009

Los tiempos cuadran ya que Bucle1 tiene menor complejidad.

n	TBucle3(t3)	TBucle2(t2)	t3/t2
100	1	1	1
200	9	1	9
400	19	14	1,3571
800	83	36	2,3056
1600	286	116	2,4655
3200	1093	304	3,5954
6400	5009	1096	4,5702
12800	30103	4859	6,1953
25600	100245	27209	3,6842
51200	372059	101758	3,6563

Los tiempos concuerdan con la complejidad, Bucle3 tiene mayor complejidad.

Bucle4-Python:

TIEMPO= MILISEGUNDOS

```
n= 100 **tiempo= 8 **contador= 171700
n= 200 **tiempo= 88 **contador= 1353400
n= 400 **tiempo= 359 **contador= 10746800
n= 800 **tiempo= 2092 **contador= 85653600
n= 1600 **tiempo= 17090 **contador= 683947200
n= 3200 **tiempo= 155883 **contador= 5466454400
```

n	Bucle4-Python	Bucle4-Java	tPython/tJava
100	8	2	4
200	88	15	5,8667
400	359	71	5,0564
800	2092	451	4,6386
1600	17090	3255	5,2504
3200	155883	29199	5,3386
6400	FdT	223960	FdT

Los tiempos concuerdan con lo esperado. Python tarda más para hacer el mismo bucle que Java porque tiene que hacer más trabajo interno.

Práctica 2

Burbuja

n	T ordenado	T inverso	T aleatorio
10000	502	1361	918
20000	1783	3704	3161
40000	6903	15285	12464
80000	27607	55970	53973
160000	112233	2225407	225559
320000	FdT	FdT	FdT
640000	FdT	FdT	FdT
1280000	FdT	FdT	FdT

T Ordenado:

```
n=10000 Tiempo=502 Repeticiones=1
n=20000 Tiempo=1783 Repeticiones=1
n=40000 Tiempo=6903 Repeticiones=1
n=80000 Tiempo=27607 Repeticiones=1
n=160000 Tiempo=112233 Repeticiones=1
```

T Inverso:

```
n=10000 Tiempo=1361 Repeticiones=1
n=20000 Tiempo=3704 Repeticiones=1
n=40000 Tiempo=15285 Repeticiones=1
n=80000 Tiempo=55970 Repeticiones=1
n=160000 Tiempo=2225407 Repeticiones=1
```

T Aleatorio:

```
n=10000 Tiempo=918 Repeticiones=1
n=20000 Tiempo=3161 Repeticiones=1
n=40000 Tiempo=12464 Repeticiones=1
n=80000 Tiempo=53973 Repeticiones=1
n=160000 Tiempo=225559 Repeticiones=1
```

El algoritmo de burbuja tiene como complejidad $O(n^2)$ por lo que si se duplica la n de entrada el tiempo se multiplica aproximadamente por 4. Los tiempos son distintos en el caso de vector ordenado porque el algoritmo no realiza

intercambios, solo comparaciones. El fundamento teórico en el que se basa lo anterior es:

$$O(n^c) \Rightarrow t_2 = (K^c) * t_1$$

Selección

n	T Ordenado	T invertido	T aleatorio
10000	943	979	482
20000	1728	1927	1789
40000	6814	7388	7459
80000	28197	29641	28637
160000	115342	117865	111728
320000	FdT	FdT	FdT
640000	FdT	FdT	FdT
1280000	FdT	FdT	FdT

T Ordenado:

```
n=10000 Tiempo=943 Repeticiones=1
n=20000 Tiempo=1728 Repeticiones=1
n=40000 Tiempo=6814 Repeticiones=1
n=80000 Tiempo=28197 Repeticiones=1
n=160000 Tiempo=115342 Repeticiones=1
```

T Invertido:

```
n=10000 Tiempo=979 Repeticiones=1
n=20000 Tiempo=1927 Repeticiones=1
n=40000 Tiempo=7388 Repeticiones=1
n=80000 Tiempo=29641 Repeticiones=1
n=160000 Tiempo=117865 Repeticiones=1
```

T Aleatorio:

```
n=10000 Tiempo=482 Repeticiones=1
n=20000 Tiempo=1798 Repeticiones=1
n=40000 Tiempo=7459 Repeticiones=1
n=80000 Tiempo=28637 Repeticiones=1
n=160000 Tiempo=111728 Repeticiones=1
```

El algoritmo de selección tiene complejidad $O(n^2)$ por lo que si se duplica la n de entrada el tiempo se multiplica aproximadamente por 4. El fundamento teórico en el que se basa lo anterior es:

$$O(n^c) \Rightarrow t_2 = (K^c) * t_1$$

Inserción

n	T Ordenado	T Inverso	T Aleatorio
10000	0	952	648
20000	0	2256	1231
40000	7	8917	4523
80000	0	35729	18602
160000	1	141721	72643
320000	8	FdT	FdT
640000	14	FdT	FdT
1280000	15	FdT	FdT

T Ordenado:

n=10000	Tiempo=0	Repeticiones=1
n=20000	Tiempo=0	Repeticiones=1
n=40000	Tiempo=7	Repeticiones=1
n=80000	Tiempo=0	Repeticiones=1
n=160000	Tiempo=1	Repeticiones=1
n=320000	Tiempo=8	Repeticiones=1
n=640000	Tiempo=14	Repeticiones=1
n=1280000	Tiempo=15	Repeticiones=1
n=2560000	Tiempo=49	Repeticiones=1
n=5120000	Tiempo=86	Repeticiones=1
n=10240000	Tiempo=145	Repeticiones=1
n=20480000	Tiempo=270	Repeticiones=1
n=40960000	Tiempo=620	Repeticiones=1
n=81920000	Tiempo=1116	Repeticiones=1

T Inverso:

n=10000	Tiempo=952	Repeticiones=1
n=20000	Tiempo=2256	Repeticiones=1
n=40000	Tiempo=8917	Repeticiones=1
n=80000	Tiempo=35729	Repeticiones=1
n=160000	Tiempo=141721	Repeticiones=1

T Aleatorio:

```

n=10000 Tiempo=648 Repeticiones=1
n=20000 Tiempo=1231 Repeticiones=1
n=40000 Tiempo=4523 Repeticiones=1
n=80000 Tiempo=18602 Repeticiones=1
n=160000 Tiempo=72643 Repeticiones=1

```

El algoritmo de inserción tiene distintas complejidades dependiendo si el vector está ordenado o no. Este algoritmo tiene complejidad $O(n)$ para caso mejor y $O(n^2)$ para caso medio o caso peor. Para ambos casos se cumple:

$$O(n^c) \Rightarrow t_2 = (K^c) * t_1$$

Con el vector ordenado ($c = 1$) (y cuando los tiempos se estabilizan, se puede ver en la captura de pantalla) si las n se duplican el tiempo también lo hace.

Para los otros dos casos ($c=2$) si las n se duplican el tiempo se multiplican por 4 aproximadamente.

Rápido

N	T Ordenado	T Inverso	T aleatorio
10000	8	5	9
20000	14	5	19
40000	25	15	25
80000	30	18	48
160000	56	57	96
320000	121	138	188
640000	260	303	333
1280000	454	613	712

T Ordenado:

```

n=10000 Tiempo=8 Repeticiones=1
n=20000 Tiempo=14 Repeticiones=1
n=40000 Tiempo=25 Repeticiones=1
n=80000 Tiempo=30 Repeticiones=1
n=160000 Tiempo=56 Repeticiones=1
n=320000 Tiempo=121 Repeticiones=1
n=640000 Tiempo=260 Repeticiones=1
n=1280000 Tiempo=454 Repeticiones=1
n=2560000 Tiempo=916 Repeticiones=1
n=5120000 Tiempo=1885 Repeticiones=1

```

T Inverso:

n=10000	Tiempo=5	Repeticiones=1
n=20000	Tiempo=5	Repeticiones=1
n=40000	Tiempo=15	Repeticiones=1
n=80000	Tiempo=18	Repeticiones=1
n=160000	Tiempo=57	Repeticiones=1
n=320000	Tiempo=138	Repeticiones=1
n=640000	Tiempo=303	Repeticiones=1
n=1280000	Tiempo=613	Repeticiones=1

T Aleatorio:

n=10000	Tiempo=9	Repeticiones=1
n=20000	Tiempo=19	Repeticiones=1
n=40000	Tiempo=25	Repeticiones=1
n=80000	Tiempo=48	Repeticiones=1
n=160000	Tiempo=96	Repeticiones=1
n=320000	Tiempo=188	Repeticiones=1
n=640000	Tiempo=333	Repeticiones=1
n=1280000	Tiempo=712	Repeticiones=1

El algoritmo rápido tiene distintas complejidades, caso peor $O(n^2)$ y $O(n \log(n))$ caso medio y caso mejor. A pesar de esto, para los tres vectores el caso peor no se produce por lo que usaremos $O(n \log(n))$. Con los tiempos medidos vemos que se cumplen:

$$O(n_1) \Rightarrow t_2 = f(n_2)/f(n_1) * t_1$$

Siendo $f(n_1) = n \log(n)$ y $f(n_2) = 2n \log(2n)$, obtenemos tras operar:

$$t_2 = 2 * (1 + \log(2)/\log(n)) * t_1$$

De aquí podemos decir que a medida que crece t_2 se acerca cada vez más a ser el doble de t_1 .

Proyecte, a partir de las complejidades y los datos de las tablas anteriores, ¿cuántos días (1 día= 86.400.000 milisegundos) tardaría cada uno de los cuatro métodos (Burbuja, Selección, Inserción y Rápido) en ordenar un vector de $n= 811.920.000$ (o sea, $n=80000 \cdot 2^4 \cdot 2^6=80000 \cdot 210$), en el caso inicialmente en orden aleatorio?

Burbuja:

Tomamos $n_1 = 160.000$, $t_1 = 225.559$ y $n_2 = 86.400.000$. Como la complejidad es $O(n^2)$ y sabemos que $O(n^2) \Rightarrow t_2 = (K^2) * t_1$ y $k = n_2/n_1$. Podemos obtener tras operar que $t_2 = 5074^2 * 225.559 = 5,8071 * 10^{12}$ milisegundos = 67.212 días.

Selección:

Tomamos $n_1 = 160.000$, $t_1 = 111.728$ y $n_2 = 86.400.000$. Como la complejidad es $O(n^2)$ y sabemos que $O(n^2) \Rightarrow t_2 = (K^2) * t_1$ y $k = n_2/n_1$. Podemos obtener tras operar que $t_2 = 5074^2 * 111.728 = 2,876 * 10^{12}$ milisegundos = 33.292 días.

Inserción:

Tomamos $n_1 = 160.000$, $t_1 = 72.643$ y $n_2 = 86.400.000$. Como la complejidad es $O(n^2)$ y sabemos que $O(n^2) \Rightarrow t_2 = (K^2) * t_1$ y $k = n_2/n_1$. Podemos obtener tras operar que $t_2 = 5074^2 * 72.643 = 1,87 * 10^{12}$ milisegundos = 21.646 días.

Rápido:

Tomamos $n_1 = 1.280.000$, $t_1 = 712$ y $n_2 = 86.400.000$. Como la complejidad es $O(n * \log(n))$ y sabemos que $O(n * \log(n)) \Rightarrow t_2 = (n_2 * \log(n_2)) / (n_1 * \log(n_1)) * t_1$ y $k = n_2/n_1$. Podemos obtener tras operar que $t_2 = 62.455$ milisegundos = $7,22 * 10^{-4}$ días.

Rápido vs inserción

n	T rápido	T inserción	T rápido / T inserción
10	280	131	2,14
15	327	150	2,18
20	280	203	1,38
25	334	307	1,09
30	487	413	1,18
35	632	493	1,28
40	681	667	1,02
45	699	810	0,86
50	967	987	0,98
55	1162	1170	0,99
60	2159	1366	1,58
65	1286	1596	0,81
70	1422	1923	0,74
75	1463	2115	0,69
80	1576	2331	0,68
85	1737	2608	0,67
90	1837	2915	0,63
95	1775	3214	0,55
100	2021	3580	0,56

Rápido

n=10	Tiempo=280	Repeticiones=100000
n=15	Tiempo=327	Repeticiones=100000
n=20	Tiempo=280	Repeticiones=100000
n=25	Tiempo=334	Repeticiones=100000
n=30	Tiempo=487	Repeticiones=100000
n=35	Tiempo=632	Repeticiones=100000
n=40	Tiempo=681	Repeticiones=100000
n=45	Tiempo=699	Repeticiones=100000
n=50	Tiempo=967	Repeticiones=100000
n=55	Tiempo=1162	Repeticiones=100000
n=60	Tiempo=2159	Repeticiones=100000
n=65	Tiempo=1286	Repeticiones=100000
n=70	Tiempo=1422	Repeticiones=100000
n=75	Tiempo=1463	Repeticiones=100000
n=80	Tiempo=1576	Repeticiones=100000
n=85	Tiempo=1737	Repeticiones=100000
n=90	Tiempo=1837	Repeticiones=100000
n=95	Tiempo=1775	Repeticiones=100000
n=100	Tiempo=2021	Repeticiones=100000

Inserción

n=10	Tiempo=131	Repeticiones=100000
n=15	Tiempo=150	Repeticiones=100000
n=20	Tiempo=203	Repeticiones=100000
n=25	Tiempo=307	Repeticiones=100000
n=30	Tiempo=413	Repeticiones=100000
n=35	Tiempo=493	Repeticiones=100000
n=40	Tiempo=667	Repeticiones=100000
n=45	Tiempo=810	Repeticiones=100000
n=50	Tiempo=987	Repeticiones=100000
n=55	Tiempo=1170	Repeticiones=100000
n=60	Tiempo=1366	Repeticiones=100000
n=65	Tiempo=1596	Repeticiones=100000
n=70	Tiempo=1923	Repeticiones=100000
n=75	Tiempo=2115	Repeticiones=100000
n=80	Tiempo=2331	Repeticiones=100000
n=85	Tiempo=2608	Repeticiones=100000
n=90	Tiempo=2915	Repeticiones=100000
n=95	Tiempo=3214	Repeticiones=100000
n=100	Tiempo=3580	Repeticiones=100000

Rápido vs Inserción

n	T rápido	T inserción	T rápido / T inserción	T Combinación
10	280	131	2,14	138
15	327	150	2,18	153
20	280	203	1,38	280
25	334	307	1,09	349
30	487	413	1,18	453
35	632	493	1,28	524
40	681	667	1,02	673
45	699	810	0,86	851
50	967	987	0,98	882
55	1162	1170	0,99	943
60	2159	1366	1,58	1081
65	1286	1596	0,81	1251
70	1422	1923	0,74	1321
75	1463	2115	0,69	1447
80	1576	2331	0,68	1640
85	1737	2608	0,67	1528
90	1837	2915	0,63	1807
95	1775	3214	0,55	1840
100	2021	3580	0,56	2168

Combinación

```

n=10      Tiempo=138      Repeticiones=100000
n=15      Tiempo=153      Repeticiones=100000
n=20      Tiempo=280      Repeticiones=100000
n=25      Tiempo=349      Repeticiones=100000
n=30      Tiempo=453      Repeticiones=100000
n=35      Tiempo=524      Repeticiones=100000
n=40      Tiempo=673      Repeticiones=100000
n=45      Tiempo=851      Repeticiones=100000
n=50      Tiempo=882      Repeticiones=100000
n=55      Tiempo=943      Repeticiones=100000
n=60      Tiempo=1081     Repeticiones=100000
n=65      Tiempo=1251     Repeticiones=100000
n=70      Tiempo=1321     Repeticiones=100000
n=75      Tiempo=1447     Repeticiones=100000
n=80      Tiempo=1640     Repeticiones=100000
n=85      Tiempo=1528     Repeticiones=100000
n=90      Tiempo=1807     Repeticiones=100000
n=95      Tiempo=1840     Repeticiones=100000
n=100     Tiempo=2168     Repeticiones=100000

```

Como se ve en la tabla, hasta el valor de n 40, el algoritmo de inserción es más eficiente que el rápido, pero a partir de ahí el algoritmo rápido es mejor.

Práctica 3

Sustraccion1

```
n=1000**TIEMPO=0**nVeces=1
n=2000**TIEMPO=0**nVeces=1
n=4000**TIEMPO=0**nVeces=1
n=8000**TIEMPO=2**nVeces=1
Exception in thread "main" java.lang.StackOverflowError
    at p3.Sustraccion1.rec1(Sustraccion1.java:9)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
```

La complejidad de Sustraccion1 es $O(n)$, a partir de $n = 8000$ el algoritmo deja de dar tiempo debido a que hay demasiadas llamadas recursivas.

Sustraccion2

```
n=1000**TIEMPO=7**nVeces=1
n=2000**TIEMPO=40**nVeces=1
n=4000**TIEMPO=137**nVeces=1
n=8000**TIEMPO=475**nVeces=1
Exception in thread "main" java.lang.StackOverflowError
    at p3.Sustraccion2.rec2(Sustraccion2.java:10)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
```

La complejidad de Sustraccion2 es $O(n^2)$, a partir de $n = 8000$ el algoritmo deja de dar tiempo debido a lo mismo que el de Sustraccion1, es decir, porque hay demasiadas llamadas recursivas.

Sustracción 3

```
n=20**TIEMPO=51**nVeces=1
n=21**TIEMPO=93**nVeces=1
n=22**TIEMPO=153**nVeces=1
n=23**TIEMPO=305**nVeces=1
n=24**TIEMPO=620**nVeces=1
n=25**TIEMPO=1234**nVeces=1
n=26**TIEMPO=2450**nVeces=1
n=27**TIEMPO=4965**nVeces=1
n=28**TIEMPO=9858**nVeces=1
n=29**TIEMPO=27727**nVeces=1
n=30**TIEMPO=68682**nVeces=1
```

La complejidad temporal de Sustraccion3 es $O(2^n)$ ya que $a=2$, $b=1$. Concuerda si comparamos con tiempos de los anteriores algoritmos. En el anterior vemos que los tiempos crecen rápidamente para los primeros valores de n mientras que en este los tiempos son todavía pequeños en las n menores, pero luego crece muy rápido.

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción3 para $n=80$? Razonar la respuesta.

Se necesita el tiempo que quieres calcular (80) y luego uno de base (25). Tras ejecutar $T * 2^{(80 - n)} = 1,2 * 2^{(80 - 25)} = 4,3234 * 10^{16}$ segundos = 1.370.784.921 años

Sustraccion4

n	Tiempo
100	0
200	28
400	168
800	1235
1600	9890
3200	83977

```
n=100**TIEMPO=0**nVeces=1
n=200**TIEMPO=28**nVeces=1
n=400**TIEMPO=168**nVeces=1
n=800**TIEMPO=1235**nVeces=1
n=1600**TIEMPO=9890**nVeces=1
n=3200**TIEMPO=83977**nVeces=1
```

Sustraccion5

n	Tiempo
30	329
32	945
34	2830
36	12682
38	43650
40	88645

```
n=30**TIEMPO=329**nVeces=1
n=32**TIEMPO=945**nVeces=1
n=34**TIEMPO=2830**nVeces=1
n=36**TIEMPO=12682**nVeces=1
n=38**TIEMPO=43650**nVeces=1
n=40**TIEMPO=88645**nVeces=1
```

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustraccion5 para $n = 80$? Razonar la respuesta.

Se necesita el tiempo que quieres calcular (80) y luego uno de base (34). Tras ejecutar $T * 3^{((80 - n))/2} = 2,83 * 3^{((80 - 34)/2)} = 2,6642 * 10^{11}$ segundos = 8448 años

Division1

A = 1

B = 3

K = 1 -> $O(n)$

```
n=1000000**TIEMPO=14**nVeces=1
n=2000000**TIEMPO=39**nVeces=1
n=4000000**TIEMPO=53**nVeces=1
n=8000000**TIEMPO=94**nVeces=1
n=16000000**TIEMPO=153**nVeces=1
n=32000000**TIEMPO=335**nVeces=1
n=64000000**TIEMPO=701**nVeces=1
n=128000000**TIEMPO=1326**nVeces=1
```

Como el algoritmo tiene complejidad $O(n)$, si se doblan las n , también los tiempos aproximadamente.

Division2

A=2

B=2

K = 1 -> $O(n \cdot \log n)$

```
n=1000000**TIEMPO=191**nVeces=1
n=2000000**TIEMPO=358**nVeces=1
n=4000000**TIEMPO=740**nVeces=1
n=8000000**TIEMPO=1538**nVeces=1
n=16000000**TIEMPO=3146**nVeces=1
n=32000000**TIEMPO=6579**nVeces=1
n=64000000**TIEMPO=13502**nVeces=1
n=128000000**TIEMPO=31238**nVeces=1
```

Comparando los tiempos con los Division1 y Division3 vemos que son mayores ya que la complejidad es mayor.

Division3

A =2

B=2

K=0 -> $O(n)$

```
n=1000000**TIEMPO=43**nVeces=1
n=2000000**TIEMPO=93**nVeces=1
n=4000000**TIEMPO=186**nVeces=1
n=8000000**TIEMPO=370**nVeces=1
n=16000000**TIEMPO=707**nVeces=1
n=32000000**TIEMPO=1148**nVeces=1
n=64000000**TIEMPO=2328**nVeces=1
n=128000000**TIEMPO=4710**nVeces=1
```

Como el algoritmo tiene complejidad lineal, si se doblan las n, también los tiempos aproximadamente.

División4

n	Tiempos
1000	16
2000	55
4000	163
8000	630
16000	2490
32000	10045

```
n=1000**TIEMPO=16**nVeces=1
n=2000**TIEMPO=55**nVeces=1
n=4000**TIEMPO=163**nVeces=1
n=8000**TIEMPO=630**nVeces=1
n=16000**TIEMPO=2490**nVeces=1
n=32000**TIEMPO=10045**nVeces=1
```

División5

n	Tiempos
1000	36
2000	128
4000	434
8000	1730
16000	7082
32000	27148

```
n=1000**TIEMPO=36**nVeces=1
n=2000**TIEMPO=128**nVeces=1
n=4000**TIEMPO=434**nVeces=1
n=8000**TIEMPO=1730**nVeces=1
n=16000**TIEMPO=7082**nVeces=1
n=32000**TIEMPO=27148**nVeces=1
```

VectorSuma:

```

METODO1: SUMA= 293
METODO2: SUMA= 293
METODO3: SUMA= 293
METODO1 >>> n= 100 ** TIEMPO= 670 NANOSEGUNDOS
METODO2 >>> n= 100 ** TIEMPO= 1424 NANOSEGUNDOS
METODO3 >>> n= 100 ** TIEMPO= 2678 NANOSEGUNDOS

```

método	n/SUMA	Tiempo(nanosegundo)
METODO1	293	
METODO2	293	
METODO3	293	
METODO1	100	670
METODO2	100	1424
METODO3	100	2678

Los tres algoritmos de VectorSuma tienen complejidad $O(n)$ aunque son de distintos tipos. Metodo1, el más eficiente, es un algoritmo iterativo mientras que los otros dos son recursivos. Metodo2 es recursivo y de sustracción mientras que Metodo3 es de división. Cuántas más llamadas recursivas se producen en el algoritmo, menos eficiente es.

Fibonacci:

```

METODO1: para n= 40 es= 102334155
METODO2: para n= 40 es= 102334155
METODO3: para n= 40 es= 102334155
METODO4: para n= 40 es= 102334155
METODO1 >>> n= 40**TIEMPO= 289 NANOSEGUNDOS
METODO2 >>> n= 40**TIEMPO= 426 NANOSEGUNDOS
METODO3 >>> n= 40**TIEMPO= 529 NANOSEGUNDOS
METODO4 >>> n= 40**TIEMPO= 4412 MILISEGUNDOS

```

Método	n	Resultado/Tiempo(nanosegundos)
METODO1	40	102334155
METODO2	40	102334155
METODO3	40	102334155
METODO4	40	102334155
METODO1	40	289
METODO2	40	426

METODO3	40	529
METODO4	40	4412

Los métodos Metodo1 y Metodo2 son iterativos mientras que los otros son recursivos. La diferencia entre ambos es que Metodo2 usa vectores mientras que Metodo1 no, lo que lo hace menos eficiente. Por otro lado, Metodo3 y Metodo4 son recursivos y de sustracción. Lo que los diferencian es la complejidad ya que la de Metodo3 es $O(n)$ mientras que la de Metodo4 es $(1,6 \cdot n)$ lo que lo hace menos eficiente. Lo ya dicho se comprueba en los tiempos medidos.

PRIMERA PARTE

```
n=1024 **TIEMPO=83** nVeces=1
n=2048 **TIEMPO=38** nVeces=1
n=4096 **TIEMPO=58** nVeces=1
n=8192 **TIEMPO=216** nVeces=1
n=16384 **TIEMPO=629** nVeces=1
n=32768 **TIEMPO=2390** nVeces=1
n=65536 **TIEMPO=9428** nVeces=1
n=131072 **TIEMPO=38149** nVeces=1
```

n	Tiempo
1024	83
2048	38
4096	58
8192	216
16384	629
32768	2390
65536	9428
131072	38149

La complejidad del algoritmo es $O(n^2)$, los tiempos concuerdan con ella.

PARTE 2

```

n=1024  **TIEMPO=9**  nVeces=1
n=2048  **TIEMPO=2**  nVeces=1
n=4096  **TIEMPO=7**  nVeces=1
n=8192  **TIEMPO=6**  nVeces=1
n=16384 **TIEMPO=8**  nVeces=1
n=32768 **TIEMPO=34** nVeces=1
n=65536 **TIEMPO=14** nVeces=1
n=131072 **TIEMPO=38** nVeces=1
n=262144 **TIEMPO=77** nVeces=1
n=524288 **TIEMPO=87** nVeces=1
n=1048576 **TIEMPO=236** nVeces=1
n=2097152 **TIEMPO=380** nVeces=1
n=4194304 **TIEMPO=741** nVeces=1

```

n	Tiempos
1024	9
2048	2
4096	7
8192	6
16384	8
32768	34
65536	14
131072	38
262144	77
524288	87
1048576	236
2097152	380
4194304	741

La complejidad del algoritmo es $O(n \cdot \log n)$. Como se puede ver los tiempos obtenidos son muy inferiores a los del algoritmo anterior ya que este es mucho más eficiente que el otro.

Práctica 4

En java:

(Usando 50 repeticiones)

n	Tiempo(ms)
8	6
16	7
32	9
64	10
128	15
256	14
512	22
1024	40
2048	76
4096	137
8192	244
16384	567
32768	1126
65536	2484

La complejidad del algoritmo es $O(n)$ y los tiempos se corresponden con ella, ya que si se duplica la n , también lo hace el tiempo aproximadamente.

```
n=8 **TIEMPO=6** nVeces=50
n=16 **TIEMPO=7** nVeces=50
n=32 **TIEMPO=9** nVeces=50
n=64 **TIEMPO=10** nVeces=50
n=128 **TIEMPO=15** nVeces=50
n=256 **TIEMPO=14** nVeces=50
n=512 **TIEMPO=22** nVeces=50
n=1024 **TIEMPO=40** nVeces=50
n=2048 **TIEMPO=76** nVeces=50
n=4096 **TIEMPO=137** nVeces=50
n=8192 **TIEMPO=244** nVeces=50
n=16384 **TIEMPO=567** nVeces=50
n=32768 **TIEMPO=1126** nVeces=50
n=65536 **TIEMPO=2484** nVeces=50
```

En Python:

(Usando 50 repeticiones)

n	Tiempo(ms)
8	0
16	4
32	0
64	0
128	13
256	18
512	34
1024	68
2048	143
4096	280
8192	625
16384	1320
32768	2826
65536	6080

La complejidad del algoritmo es $O(n)$ y los tiempos se corresponden con ella, ya que si se duplica la n , también lo hace el tiempo aproximadamente.

```
n=8 **TIEMPO=0**nVeces=50
n=16 **TIEMPO=4**nVeces=50
n=32 **TIEMPO=0**nVeces=50
n=64 **TIEMPO=0**nVeces=50
n=128 **TIEMPO=13**nVeces=50
n=256 **TIEMPO=18**nVeces=50
n=512 **TIEMPO=34**nVeces=50
n=1024 **TIEMPO=68**nVeces=50
n=2048 **TIEMPO=143**nVeces=50
n=4096 **TIEMPO=280**nVeces=50
n=8192 **TIEMPO=625**nVeces=50
n=16384 **TIEMPO=1320**nVeces=50
n=32768 **TIEMPO=2826**nVeces=50
n=65536 **TIEMPO=6080**nVeces=50
```

Práctica 5

1. Definición de estados:

En el código i representa el número de vehículos y l la unidad de longitud en babor. La longitud de estribor no se almacena ya que es $\text{sumatorio}[i] - l$.

2. Relación de recurrencia:

Para cada nuevo coche que se introduce, se generan cambios en dp (siendo v_i la longitud del vehículo y si suma de los primeros i vehículos):

- Si se coloca en babor, se incrementa la ocupación:
 $dp[i][l+v_i] = \text{true}$ si $l + v_i \leq \text{Longitud de babor}$
- Si se coloca en estribor, se mantiene la ocupación en babor:
 $dp[i][l] = \text{true}$ si $l - v_i \leq \text{Longitud de babor}$

3. Complejidad

La complejidad del algoritmo es $O(n \cdot l)$ siendo n el número de vehículos que se introducen en el barco y l la longitud del barco.

Práctica 6

Debido a la gran diferencia de tiempos entre los primeros 8 ficheros y los dos últimos voy a hacer dos tablas. La primera es con una repetición por lo que los tiempos de los primeros son 0 pero solo interesan los de los dos últimos. La segunda tabla es para comparar los tiempos de los primeros ficheros.

Tabla 1 – 1 Repetición

```
PS C:\Users\samue\Desktop\alg_TobiasMenendezSamuelUO305955\p6> java AlmacenajeContenedoresTiempos 1
CasosPrueba/test00.txt **TIEMPO=2** nVeces=1
CasosPrueba/test01.txt **TIEMPO=0** nVeces=1
CasosPrueba/test02.txt **TIEMPO=0** nVeces=1
CasosPrueba/test03.txt **TIEMPO=0** nVeces=1
CasosPrueba/test04.txt **TIEMPO=0** nVeces=1
CasosPrueba/test05.txt **TIEMPO=0** nVeces=1
CasosPrueba/test06.txt **TIEMPO=0** nVeces=1
CasosPrueba/test07.txt **TIEMPO=0** nVeces=1
CasosPrueba/test08.txt **TIEMPO=180** nVeces=1
CasosPrueba/test09.txt **TIEMPO=1593907** nVeces=1
```

Fichero	Tiempo(ms)
test00	2
test01	0
...	...
test08	180
test09	1593907

Tabla 2 – 10000000 (Ficheros no ordenados por tamaño)

```
PS C:\Users\samue\Desktop\alg_TobiasMenendezSamuelUO305955\p6> java AlmacenajeContenedoresTiempos 10000000
CasosPrueba/test00.txt **TIEMPO=2187** nVeces=10000000
CasosPrueba/test01.txt **TIEMPO=2816** nVeces=10000000
CasosPrueba/test02.txt **TIEMPO=6041** nVeces=10000000
CasosPrueba/test03.txt **TIEMPO=4101** nVeces=10000000
CasosPrueba/test04.txt **TIEMPO=14360** nVeces=10000000
CasosPrueba/test05.txt **TIEMPO=15370** nVeces=10000000
CasosPrueba/test06.txt **TIEMPO=5089** nVeces=10000000
CasosPrueba/test07.txt **TIEMPO=5979** nVeces=10000000
```

Fichero	Tiempo(ms)
test00	2187
test01	2816
test02	6041
test03	4101
test04	14360
test05	15370
test06	5089
test07	5979
test08	FdT
Test09	FdT

La complejidad del algoritmo es $O(O!)$. El algoritmo explora todas las posibles asignaciones de objetos a los contenedores. A medida que avanza el algoritmo, el número de opciones aumenta ya que por cada nuevo objeto puede asignarse a más configuraciones. Además, se puede ver que, aumentando el tamaño, el tiempo crece muy rápidamente.

Práctica 7

A diferencia de la p6, ahora no hay tanta diferencia de tiempos entre los distintos ficheros por la nueva PODA. Así, para n = 1000000:

```
PS C:\Users\samue\Desktop\alg_TobiasMenendezSamuelUO305955\p7> java AlmacenajeContenedoresRyPTiempos 1000000
test00.txt **TIEMPO=646** nVeces=1000000
test01.txt **TIEMPO=680** nVeces=1000000
test02.txt **TIEMPO=1095** nVeces=1000000
test03.txt **TIEMPO=840** nVeces=1000000
test04.txt **TIEMPO=1882** nVeces=1000000
test05.txt **TIEMPO=3057** nVeces=1000000
test06.txt **TIEMPO=943** nVeces=1000000
test07.txt **TIEMPO=1162** nVeces=1000000
test08.txt **TIEMPO=6255** nVeces=1000000
test09.txt **TIEMPO=16846** nVeces=1000000
```

Fichero	Tiempo(ms)
test00	646
test01	680
test02	1095
test03	840
test04	1882
test05	3057
test06	943
test07	1162
test08	6255
Test09	16846

Comparándolo con los tiempos de p6:

```
PS C:\Users\samue\Desktop\alg_TobiasMenendezSamuelUO305955\p6> java AlmacenajeContenedoresTiempos 1000000
CasosPrueba/test00.txt **TIEMPO=227** nVeces=1000000
CasosPrueba/test01.txt **TIEMPO=269** nVeces=1000000
CasosPrueba/test02.txt **TIEMPO=498** nVeces=1000000
CasosPrueba/test03.txt **TIEMPO=328** nVeces=1000000
CasosPrueba/test04.txt **TIEMPO=1150** nVeces=1000000
CasosPrueba/test05.txt **TIEMPO=1194** nVeces=1000000
CasosPrueba/test06.txt **TIEMPO=400** nVeces=1000000
CasosPrueba/test07.txt **TIEMPO=473** nVeces=1000000
```

```
PS C:\Users\samue\Desktop\alg_TobiasMenendezSamuelUO305955\p6> java AlmacenajeContenedoresTiempos 1
CasosPrueba/test00.txt **TIEMPO=2** nVeces=1
CasosPrueba/test01.txt **TIEMPO=0** nVeces=1
CasosPrueba/test02.txt **TIEMPO=0** nVeces=1
CasosPrueba/test03.txt **TIEMPO=0** nVeces=1
CasosPrueba/test04.txt **TIEMPO=0** nVeces=1
CasosPrueba/test05.txt **TIEMPO=0** nVeces=1
CasosPrueba/test06.txt **TIEMPO=0** nVeces=1
CasosPrueba/test07.txt **TIEMPO=0** nVeces=1
CasosPrueba/test08.txt **TIEMPO=180** nVeces=1
CasosPrueba/test09.txt **TIEMPO=1593907** nVeces=1
```

Se aprecia que se produce una mejora drástica en los últimos dos ficheros que son los más complejos. También se nota que los ficheros más sencillos tardan un poco más porque hay una PODA más compleja y con la otra más sencilla ya era más que de sobra.